

Christen Mendis

Presenter Pack

Member D — Analysis, Documentation & Presentation

Slide 4

Maze Runner · Problem Statement 15

REVA University · 07/10/2026

SECTION 1 — The project (everyone must know this)

What we built

A maze game where the player has to reach the exit while an enemy chases them using real pathfinding. The enemy recalculates the shortest route to the player on every single step, using either BFS or A*, and you can switch between the two live and watch the difference on screen.

From level 4 a **second enemy** joins running the *other* algorithm, so BFS and A* hunt the same player at the same time.

How it works, in 60 seconds

The maze is a grid of 1s and 0s — 1 is a wall, 0 is floor. The player and each enemy sit on one cell. Sixty times a second we read the keyboard, move everything one cell if its cooldown has expired, and redraw. The enemy decides where to go by running a search from itself to the player, then stepping onto the second cell of the route it found.

Everything else is detail on top of that.

The five facts everyone must be able to say

1. **BFS uses a FIFO queue, so it finds the shortest path.** It finishes every cell at distance d before touching anything at $d+1$, so the first time it reaches the goal it got there in the fewest steps.
2. **A* scores cells** $f = g + h$ — g is steps taken, h is Manhattan distance remaining. h must never overestimate or optimality is lost.
3. **A* expanded ~26% fewer cells but ran slower** in wall-clock time, because its heap costs $O(\log n)$ per operation against the deque's $O(1)$.
4. **The enemy used to camp the exit** — "farthest from the player" is the opposite corner, which is where the exit is. Fixed by maximising `min(distance to player, distance to exit)`.
5. **We tuned difficulty by measurement, not feel** — `autoplay.py` ran 500 bot playthroughs and found the enemy was only $1.1\times$ slower than the player.

The numbers

Tech	Python 3 + pygame-ce. No dataset, no network, no hardware
Tests	226 headless checks, all passing
Benchmark	50 mazes per configuration
Balance	500 bot playthroughs; L1–3 ~68–70%, L4–10 20–38%
Repo	github.com/Hyperval/maze-runner

Who built what

Person	Owns
Akhil Sathish Kumar	Maze generation, braiding, difficulty balance
Akshay N	Pathfinding — BFS, A*, distance maps
Abdullah Subhaan K	Rendering, menu, pause, HUD, test suite
Christen Mendis	benchmark.py, autoplay.py, docs, demo video

SECTION 2 — Your code: `benchmark.py` and `autoplay.py`

You own the two tools that **measure** the project. Both are standalone — they import the game's modules but never modify them, so they can't break the demo.

`benchmark.py` — measuring BFS against A*

Generates 50 mazes at each of five "braid" levels, runs both searches from start to exit, and records cells expanded, path length and time taken.

Two Python details worth knowing:

```
matplotlib.use("Agg")    # BEFORE importing pyplot
```

"Agg" draws to a file instead of opening a window. Without it the script blocks forever waiting for you to close a plot.

```
begin = time.perf_counter()
for _ in range(20): search(...)    # repeat, because one run is too fast
```

`perf_counter` is the high-resolution clock. A single search is faster than the clock's resolution, so we time 20 and divide.

What it found

Braid	BFS cells	A* cells	A* saves	BFS ms	A* ms
0.00	183.5	163.9	10.7%	0.141	0.216
0.35	214.1	157.9	26.2%	0.169	0.219
0.80	256.3	171.9	33.0%	0.250	0.302

Finding 1: A*'s advantage depends on how open the maze is. In a tight maze it saves only ~11%, because the corridors are so constrained there's barely anything for the heuristic to discriminate between. Add loops and it climbs past 30%.

Finding 2 — the interesting one: A* expands fewer cells but takes **more wall-clock time**. Not a contradiction: - BFS uses a `deque` — push and pop are $O(1)$, a couple of pointer moves. - A* uses a **heap** — every push and pop is $O(\log n)$, plus it computes the heuristic for every neighbour.

So A* does *less* work in cells touched, but *more* work per cell. At ~650 cells the per-cell overhead dominates. On a much larger map, or where visiting a cell is expensive, A* wins decisively.

`autoplay.py` — the robot tester

Runs the game headlessly with a bot playing it, hundreds of times.

```
os.environ["SDL_VIDEODRIVER"] = "dummy"    # BEFORE importing pygame
```

Renders to nothing, so no window opens and 500 runs finish in seconds.

The bot walks the shortest route to its objective (nearest coin, then the exit) using the same BFS the enemy uses, and **routes around the enemies** by treating cells near them as temporary walls:

```
safe_grid = [row[:] for row in game.grid]    # copy each row
```

`row[:]` **makes a copy**. A plain `list(grid)` would copy the outer list but share the inner rows — writing to `safe_grid` would corrupt the real maze. This mutable-aliasing trap catches everyone once.

What it found — the story you're telling on your slide

The first run reported **levels 3–10 unwinnable, 0% win rate**. Investigated in stages:

1. The bot was ignoring enemies and walking into them. Added evasion → still only 0–20%. So the bot wasn't the whole story.

2. Coins spawned in the farthest cells, so every coin meant crossing the entire maze under pursuit. Moved them to a middle band, added a 2.2s head start → levels 1–3 reached 47–73%.
3. Levels 4+ still collapsed — exactly where the second enemy joins. Slowing it helped only slightly, so that wasn't the root cause either.
4. **Real cause:** `ENEMY_MIN_DELAY` was 100ms against the player's 90ms — only **1.1× slower** than something that never rests and always takes the shortest route. Raised to 140ms. Curve fixed.

Final over 500 runs: L1–3 ~68–70%, L4–10 20–38%, no crashes.

The honest framing: the bot is a **floor, not a ceiling**. It only dodges one cell ahead and never plans a route that keeps the enemy at distance, so a human who learns a level beats these numbers.

SECTION 3 — Your presenting job

You present **Slide 4 — "Three things that went wrong"**. Target: 75–90 seconds.

Examiners hear "integration was hard" from every team. You have three specific bugs with specific fixes and a measured chart. **This is where the team's Innovation and Technical marks come from — take your time.**

What to say

Start with the exit-camping bug — it's concrete and instantly understandable:

We placed the enemy at the cell farthest from the player. That's the opposite corner — which is exactly where the exit is. So the enemy sat on the goal and camped it, and the level was impossible. We fixed it by maximising the *smaller* of the distance to the player and the distance to the exit, using real walking distance rather than straight-line, so walls count.

Then the balance finding, and stress that a tool found it:

We then couldn't tell whether the game was actually fair. So we wrote a bot that plays it headlessly — it reported a 0% win rate from level 3. After investigating, the root cause was that our enemy's fastest speed was 100 milliseconds against the player's 90. Only 1.1 times slower than something that never rests and always takes the shortest path. We raised it, and over 500 runs levels 1 to 3 now sit around 68 to 70%.

Then deliver the blue box slowly — it's the best line in the presentation:

One result surprised us. A* expanded about 26% fewer cells, as expected. But it ran *slower* — 0.219 milliseconds against 0.169. BFS uses a deque where push and pop are constant time; A* uses a heap at $O(\log n)$ plus a heuristic per neighbour. At our maze size the overhead per cell outweighs the cells saved. On a much larger map, A* would win.

Pause after that. Let it land.

Why this slide matters

Most teams assert "A* is faster". You have a chart from 50 mazes showing the situation is more subtle, and a bug your own tooling caught. That's the difference between a claim and evidence.

Questions that come to you

- "How many runs?" — 500 playthroughs, 50 per level.
- "Is the bot as good as a human?" — no, it's a floor. It only dodges one cell ahead and doesn't plan to keep distance. A human who learns a level beats it.
- "Why does A*'s advantage depend on the maze?" — in a tight maze there's barely any choice for the heuristic to discriminate between; loops give the search real alternatives.
- "So which algorithm would you ship?" — depends on the map size and what visiting a cell costs. At our size, BFS. At scale, A*.

Your other jobs (not on stage)

- **The backup video.** You hold it. If the live demo fails, Abdullah hands to you and you play it. Have it open in a window *before* you walk in.
- **Second tester.** You didn't write the game code, which makes you best placed to break it. Try the dumb things — hold every key, spam restart, pause during a win.
- **Submission.** README accuracy, screenshots in `assets/`, team details sheet, Annexure B signed by all four.

Final checklist

- ☐ Backup video recorded, uploaded, link in README
- ☐ Backup video **open in a window** before the presentation starts
- ☐ Re-run `python benchmark.py` so the chart matches what you'll say
- ☐ Rehearse the A*-slower explanation out loud — it's the hardest to say well
- ☐ Annexure B chased from all four members